
Building Ultra-Lightweight Models for Image Tasks

Exploring the Lower Bounds of Visual Intelligence

Saptak Bhoumik
saptakbhoumik.acad@gmail.com

Abstract

Most image-based ML models today are bulky, data-hungry, and overkill for many practical tasks. This project takes the opposite approach: how small can we go while still getting the job done? We explore a range of techniques to aggressively shrink image models, from replacing learned filters with handcrafted ones to simplifying architecture designs and other clever tricks.

1. Introduction

This paper is version 3 of an ongoing project exploring how small we can make image-based machine learning models while still keeping them functional. The goal is to test the limits by stripping things down to the bare minimum and seeing what survives. Right now, we are not trying to explain why certain tricks work. That will be saved for a future version. This version is focused on what works best given hard constraints on parameter count and compute.

I have tested many different ideas. For simplicity, I limit this version to a single distinct task: basic image classification. Specifically, I work on distinguishing cats from dogs using the well-known Microsoft Dogs vs Cats dataset (<https://www.kaggle.com/c/dogs-vs-cats>) and the Cifar 10 dataset (<https://www.cs.toronto.edu/~kriz/cifar.html>). I chose this because it is simple, popular, and just complex enough to make things interesting.

All code and experiments are available in the GitHub repository: <https://github.com/SaptakBhoumik/TinyVision>. Future versions of this paper will explore other tasks, include comparisons between methods, and dig into the underlying reasons behind performance. For now, I will just document my thinking process behind choosing a method and note the best test accuracy achieved. I will also denote a list of questions that I will try to answer in the future. I admit the method I used is very simple and all of it is just my hypothesis without a mathematical basis, to the point that I doubt it will be found interesting, but I encourage reading it as an early draft of my findings.

2. Disclaimer

This is more of an experimentation log than an actual paper. You can read it but don't expect it to be complete

3. Image classification

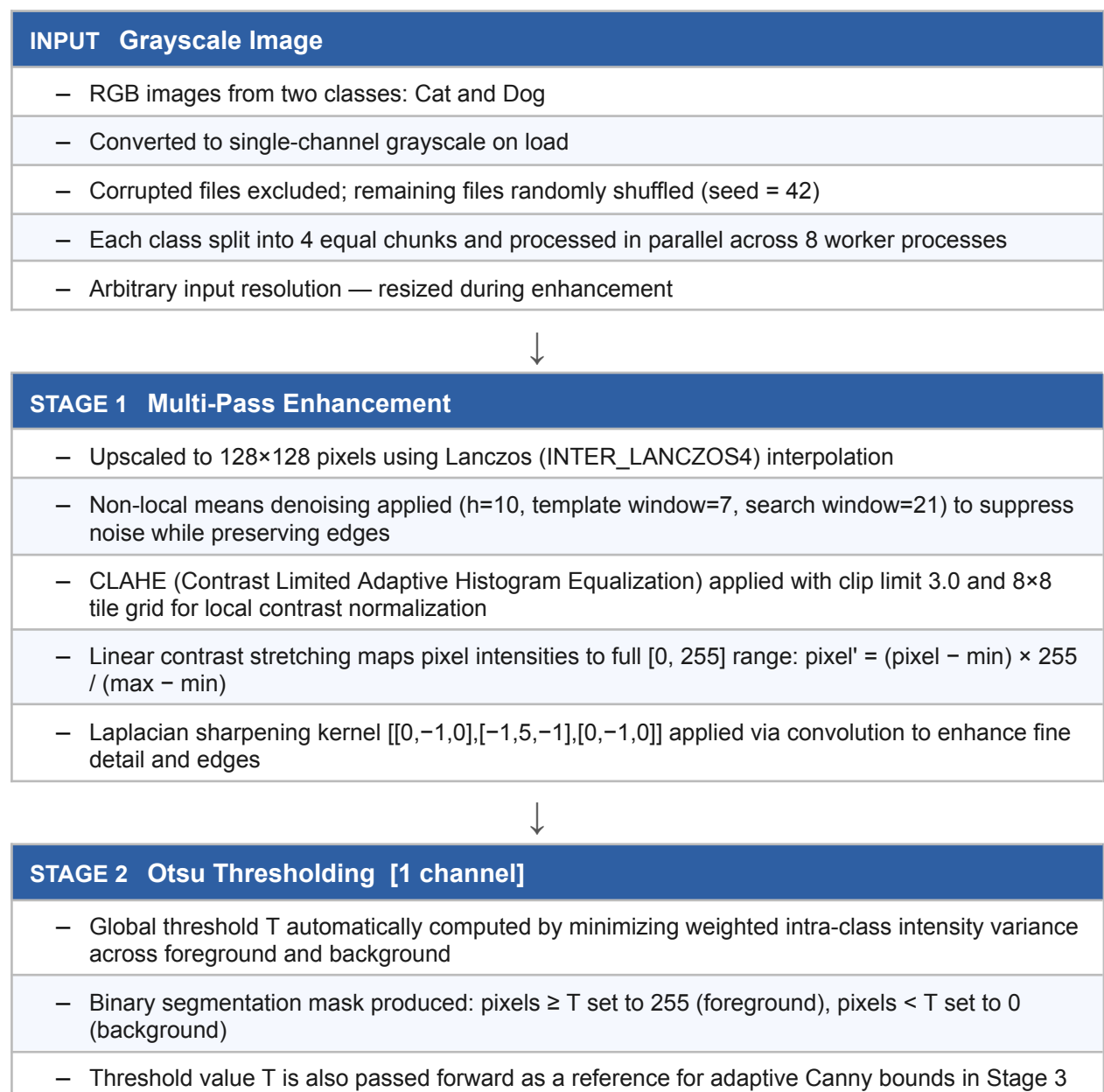
3.1 Cat vs Dog Classification:-

Main hypothesis:-

My core hypothesis for this subproject is that by incorporating a stack of predefined filters (such as edge detection and texture detection), we can introduce a stronger inductive bias. This, in turn, should allow the model to achieve the same performance with a reduced number of parameters.

The concept builds upon the inherent inductive bias present in traditional Convolutional Neural Networks (CNNs) but aims to amplify it further. The rationale behind using filters like edge detection is that many models learn these fundamental features internally anyway; by providing them explicitly, we simplify the model's learning task.

Filter Stack:-



STAGE 3 Canny Edge Detection [2 channels]

- Gaussian smoothing applied internally, then gradient magnitudes computed and non-maximum suppression performed
- Edges retained only if magnitude exceeds high threshold, or exceeds low threshold and is connected to a strong edge
- Channel 1 — Loose thresholds (0.3T, 0.7T): captures fine, weak, and subtle edges
- Channel 2 — Strict thresholds (1.2T, 1.8T): retains only dominant structural boundaries



STAGE 4 Scharr Gradient Magnitude [2 channels]

- Horizontal (Gx) and vertical (Gy) gradients computed using Scharr kernels, which have better rotational symmetry than Sobel
- Gradient magnitude combined as $\sqrt{Gx^2 + Gy^2}$ and clipped to [0, 255]
- Channel 1 — (scale=1, delta=0): standard gradient magnitude response
- Channel 2 — (scale=2, delta=0): amplified gradient, emphasizing stronger intensity transitions



STAGE 5 Local Binary Pattern (LBP) [2 channels]

- Each pixel encoded by thresholding P neighbors sampled on a circle of radius R against the center pixel value
- Uniform rotation-invariant mode used, producing texture descriptors invariant to monotonic intensity shifts
- LBP output normalized to [0, 255] for consistent representation
- Channel 1 — (P=16, R=2): fine-grained texture over a small 2-pixel neighborhood
- Channel 2 — (P=24, R=4): coarser texture structure captured over a larger 4-pixel neighborhood



STAGE 6 Gabor Filter [2 channels]

- Gabor kernel constructed as a sinusoidal wave modulated by a Gaussian envelope (kernel size=21, $\sigma=8$, $\lambda=10$, $\gamma=0.5$, $\phi=0$)
- Selectively responds to spatial frequency content at a specific orientation, mimicking early visual cortex processing
- Channel 1 — ($\theta=0^\circ$): sensitive to horizontal frequency structure and vertical edges
- Channel 2 — ($\theta=90^\circ$): sensitive to vertical frequency structure and horizontal edges



OUTPUT 10-Channel Feature Tensor

- Each image produces a 10-channel feature map stack at 128×128 resolution

- Channel composition: 1 enhanced grayscale + 1 Otsu mask + 2 Canny maps + 2 Scharr maps + 2 LBP maps + 2 Gabor maps

Training Details

Parameter	Value / Detail
Epochs	10
Optimiser	AdamW
Learning rate — Epochs 1–6	0.001
Learning rate — Epochs 7–9	0.0001
Learning rate — Epoch 10	0.00001
Batch size	40 (two models trained at batch size 16; see footnote)
Input image size	128 × 128 pixels
Augmentation	Horizontal flip applied to every image, effectively doubling the dataset. No images are shared between the training and testing splits.
Weight initialisation	Kaiming Normal (He) applied to all Conv2d and Linear layers — mode=fan_in, negative slope a=0.25 (matching PReLU default). Biases initialised to zero.
Initialisation rationale	Kaiming Normal with a=0.25 accounts for the PReLU activation used throughout the network, which behaves as a leaky ReLU with a learnable slope initialised near 0.25. This keeps gradient variance stable at network initialisation.

Architecture Description

All four models (Model0 – Model3) share the same three-stage hierarchical structure. The global architecture is described first; the per-model section that follows documents only the components that differ.

Global Architecture

Every model processes a 10-channel feature tensor through three sequential stages before producing a binary classification output.

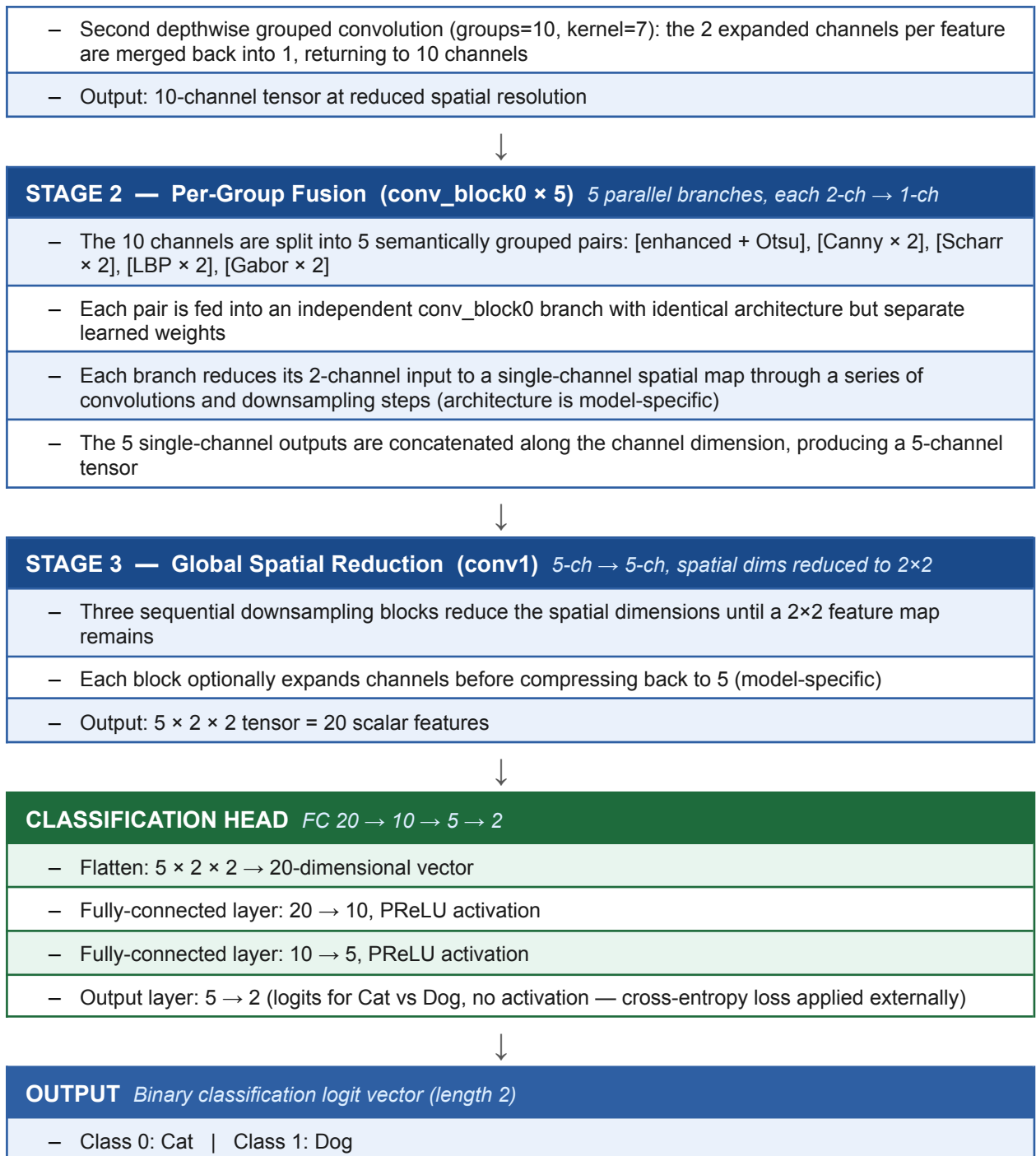
INPUT $10 \times H \times W$ feature tensor

- 10-channel stack produced by the preprocessing pipeline (enhanced grayscale, Otsu mask, 2× Canny, 2× Scharr, 2× LBP, 2× Gabor)
- Channels are normalised before entering the network (Batchnorm is used. Which is unusual, but the empirical result for my case suggested it works. Although It kinda makes sense if you think about it)



STAGE 1 — Per-Feature Extraction (conv0) $10\text{-ch} \rightarrow 20\text{-ch} \rightarrow 10\text{-ch}$, grouped convolutions

- Depthwise grouped convolution (groups=10, kernel=7): each of the 10 input channels is independently expanded to 2 channels, producing 20 feature maps — no cross-channel mixing at this step
- BatchNorm + PReLU activation applied after each convolution throughout the network
- Spatial downsampling (pooling or strided conv, model-specific) reduces resolution



Per-Model Differences

The table below summarises the design choices that distinguish each model. All other components are identical to the global architecture above.

Component	Model0	Model1	Model2	Model3
Downsampling method	Pooling layer	Stride-2 conv	Stride-2 conv	Pooling layer
conv0 downsampling	After 1st grouped conv	Stride=2 on 1st grouped conv	Stride=2 on 1st grouped conv	After 1st grouped conv

conv_block0 depth	4 convs + 2 pools	4 convs, stride-2 ×2	2 convs, stride-2 ×2	2 convs + 2 pools
conv_block0 channel expansion	2→8→4→2→1 (ratio=2)	2→8→4→2→1 (ratio=2)	2→2→1 (no expansion)	2→2→1 (no expansion)
conv1 channel expansion	5→10→5, ×3 blocks	5→10→5, ×3 blocks	5→5, no expansion	5→5, no expansion
conv1 downsampling	Pooling after expand conv	Stride=2 on expand conv	Stride=2 (3×2×2 kernels)	Pooling after each conv

Model-Level Detail

Model0 — Pooling + Channel Expansion

Downsampling is performed exclusively through pooling layers (injected via the pool argument). conv_block0 expands channels aggressively before compressing ($2 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$), controlled by ratio=2. conv1 similarly expands $5 \rightarrow 10$ before each compress step, giving the network wider intermediate representations at the cost of more parameters.

Model1 — Strided Convolutions + Channel Expansion

Structurally identical to Model0 in channel counts and expansion ratios, but all pooling layers are replaced by stride=2 convolutions. This allows the network to learn its own spatial subsampling rather than applying a fixed pooling operation, potentially retaining more spatial information.

Model2 — Strided Convolutions, No Expansion

Removes the channel expansion used in Model0 and Model1. conv_block0 reduces 2 channels directly to 1 over two strided convolutions without widening. conv1 keeps a constant 5-channel width and uses mixed kernel sizes (3×3 with stride=2, then 2×2 with stride=2 twice) for spatial reduction. This produces the most parameter-efficient of the four architectures.

Model3 — Pooling, No Expansion

Combines the pooling-based downsampling of Model0 with the simplified no-expansion channel structure of Model2. conv_block0 applies a single 3×3 conv followed by pooling twice to reduce 2 channels to 1. conv1 uses three identical Conv→BN→PReLU→Pool blocks, keeping 5 channels throughout. This is the simplest architecture by design, serving as a low-complexity baseline.

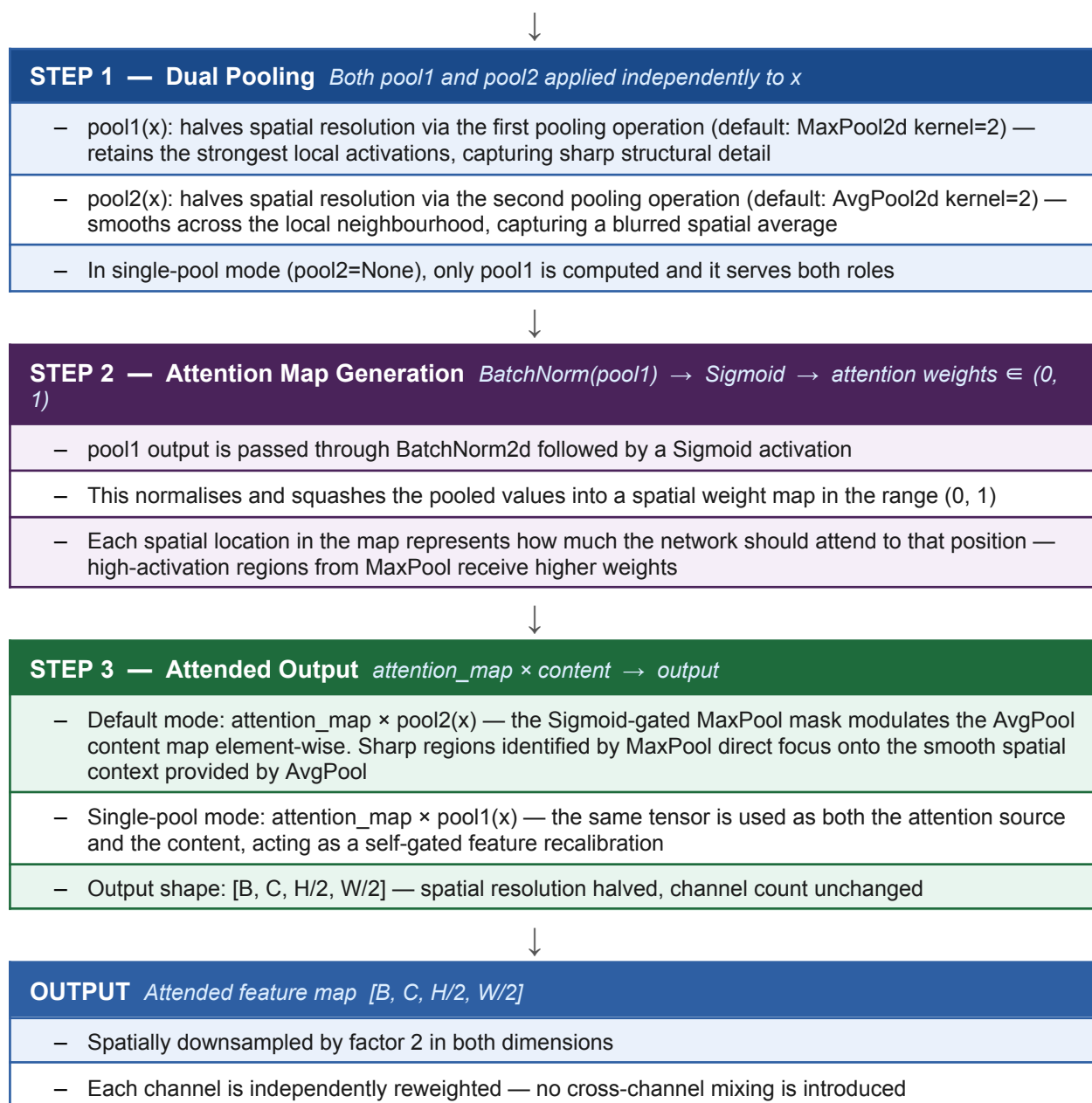
HybridPoolingV0

HybridPoolingV0 is a lightweight pseudo-attention pooling module that decouples the roles of spatial selection and content aggregation during downsampling. Rather than applying a single pooling operation uniformly across the feature map, it uses one pooled output to generate a soft spatial attention mask and a second pooled output as the content to be weighted, combining both before passing to the next layer.

Forward Pass

INPUT Feature map x of shape $[B, C, H, W]$

– B = batch size, C = channels, $H \times W$ = spatial dimensions



Pooling Configurations

The module is parameterised by the choice of pool1 and pool2, allowing several operating modes. The default Max+Avg combination reflects the hypothesis that MaxPool locates salient regions while AvgPool provides the content to be amplified at those locations.

Configuration	pool1 (attention map)	pool2 (content)
Default (recommended)	MaxPool — sharp, high-activation regions	AvgPool — smooth spatial average
Single-pool mode	MaxPool or AvgPool	None — attention applied to pool1 itself
Dual AvgPool	AvgPool	AvgPool
Dual MaxPool	MaxPool	MaxPool

Result:-

This report compares all 28 trained model variants across four dimensions: base architecture, pooling strategy, channel expansion ratio, and batch size. Derived metrics include generalisation gap (train – test accuracy), accuracy per parameter, and accuracy per MMAC.

	Test accuracy $\geq 86\%$ (top tier)
	Test accuracy 84–86% (mid tier)
	Test accuracy $< 80\%$ (below baseline)
	Generalisation gap ≤ 1.5 pp (well-generalised)
	Generalisation gap ≥ 3.5 pp (overfit risk)

1. Master Comparison Table

All 28 models. Test accuracy cells are colour-coded (green $\geq 86\%$, yellow 84–86%, red $< 80\%$). Gap cells are green ≤ 1.5 pp, red ≥ 3.5 pp.

Model	Arch	Pooling	Ratio	BS	Params (k)	MMAC	TrainAc c%	TestA cc%	Gap(pp)	TrainLo ss	TestLo ss	Acc/P aram	Acc/M MAC
model0_0	Model0	Hybrid(Max →Avg)	2	40	8.25	30.52	87.34%	85.48 %	1.86	0.2915	0.3345	10.36	2.8
model0_1	Model0	Hybrid(Max →Avg)	2	16	8.25	30.52	88.67%	86.56 %	2.11	0.2629	0.31	10.49	2.84
model1_0	Model1	Strided Conv	2	40	8.05	11.71	82.67%	79.93 %	2.74	0.3867	0.428	9.93	6.83
model2_0	Model1	Strided Conv	1	40	4.79	9.95	83.42%	81.37 %	2.05	0.3755	0.402	16.99	8.18
model3_0	Model1	Strided Conv	3	40	12.2	14.24	85.6%	81.99 %	3.61	0.3282	0.4027	6.72	5.76
model4_0	Model2	Strided Conv	1	40	3.15	9.25	78.7%	77.38 %	1.32	0.4576	0.4767	24.57	8.37
model5_0	Model0	Hybrid(Max →Avg)	1	40	4.91	26.66	84.87%	83.68 %	1.19	0.345	0.3695	17.04	3.14
model6_0	Model0	Hybrid(Max →Avg)	3	40	12.48	35.3	87.68%	85.43 %	2.25	0.2864	0.3294	6.85	2.42
model7_0	Model0	AvgPool	2	40	8.05	29.25	85.72%	83.88 %	1.84	0.3293	0.3593	10.42	2.87
model8_0	Model0	AvgPool	1	40	4.79	25.62	84.17%	82.77 %	1.4	0.3575	0.3842	17.28	3.23
model9_0	Model0	AvgPool	3	40	12.2	33.81	88.55%	85.74 %	2.81	0.2692	0.3251	7.03	2.54
model10_0	Model0	MaxPool	1	40	4.79	25.62	82.11%	78.81 %	3.3	0.3936	0.4331	16.45	3.08
model11_0	Model0	MaxPool	2	40	8.05	29.25	85.17%	83.14 %	2.03	0.3407	0.3803	10.33	2.84
model12_0	Model0	MaxPool	3	40	12.2	33.81	87.82%	84.59 %	3.23	0.2877	0.3498	6.93	2.5
model13_0	Model0	Hybrid(Avg →Max)	1	40	4.91	26.66	83.02%	81.48 %	1.54	0.378	0.4072	16.59	3.06
model14_0	Model0	Hybrid(Avg →Max)	2	40	8.25	30.52	87.46%	84.61 %	2.85	0.2945	0.3543	10.26	2.77

model15_0	Model0	Hybrid(Avg →Max)	3	40	12.48	35.3	89.65%	86.87 %	2.78	0.247	0.3078	6.96	2.46
model15_1	Model0	Hybrid(Avg →Max)	3	16	12.48	35.3	89.69%	86.06 %	3.63	0.2479	0.3213	6.9	2.44
model16_0	Model0	Hybrid(Avg → $\emptyset$)	1	40	4.91	25.83	84.19%	82.91 %	1.28	0.3545	0.381	16.89	3.21
model17_0	Model0	Hybrid(Avg → $\emptyset$)	2	40	8.25	29.51	86.28%	84.27 %	2.01	0.3178	0.3582	10.21	2.86
model18_0	Model0	Hybrid(Avg → $\emptyset$)	3	40	12.48	34.11	88.46%	85.87 %	2.59	0.2697	0.323	6.88	2.52
model19_0	Model0	Hybrid(Max → $\emptyset$)	1	40	4.91	25.83	84.4%	82.07 %	2.33	0.3538	0.4041	16.71	3.18
model20_0	Model0	Hybrid(Max → $\emptyset$)	2	40	8.25	29.51	85.55%	82.89 %	2.66	0.3311	0.3841	10.05	2.81
model21_0	Model0	Hybrid(Max → $\emptyset$)	3	40	12.48	34.11	88.53%	85.21 %	3.32	0.2692	0.3397	6.83	2.5
model22_0	Model3	Hybrid(Max →Avg)	-	40	3.5	25.07	84.54%	83.37 %	1.17	0.3522	0.3772	23.82	3.33
model23_0	Model3	AvgPool	-	40	3.4	24.14	83.2%	82.12 %	1.08	0.376	0.3941	24.15	3.4
model24_0	Model3	MaxPool	-	40	3.4	24.14	82.99%	80.78 %	2.21	0.3824	0.4108	23.76	3.35
model25_0	Model3	Hybrid(Avg →Max)	-	40	3.5	25.07	82.71%	80.73 %	1.98	0.3891	0.4189	23.07	3.22
model26_0	Model3	Hybrid(Avg → $\emptyset$)	-	40	3.5	24.32	83.77%	82.19 %	1.58	0.3693	0.3931	23.48	3.38
model27_0	Model3	Hybrid(Max → $\emptyset$)	-	40	3.5	24.32	82.62%	80.5%	2.12	0.3881	0.4258	23	3.31

2. Architecture Comparison (Model0 vs Model1 vs Model2 vs Model3)

Model0 uses pooling layers with channel expansion. Model1 replaces pooling with strided convolutions but keeps expansion. Model2 uses strided convolutions without expansion. Model3 uses pooling without expansion. Averages below span all pooling and ratio variants within each architecture.

Architecture	N	Avg Test%	Best%	Worst%	Avg Gap	Avg Params
Model0 (pool + expansion)	20	84.12%	86.87%	78.81%	2.35	8.67k
Model1 (strided conv + expansion)	3	81.10%	81.99%	79.93%	2.80	8.35k
Model2 (strided conv, no expansion)	1	77.38%	77.38%	77.38%	1.32	3.15k
Model3 (pool, no expansion)	6	81.61%	83.37%	80.50%	1.69	3.47k

Key observations:

- Model0 consistently achieves the highest average test accuracy across all pooling and ratio variants, benefiting from learnable pooling (HybridPooling) combined with channel expansion.
- Model1 (strided conv + expansion) underperforms relative to Model0 despite similar parameter budgets, suggesting that the fixed pooling operation provides a useful inductive bias that strided convolutions do not replicate at this scale.

- Model2 is the lightest architecture (3.15k params) but pays a clear accuracy cost, producing the lowest test accuracy in the entire experiment at 77.38%.
- Model3 achieves a strong accuracy-to-parameter ratio: comparable test accuracy to Model1 at roughly a third of the parameters, making it the most efficient baseline.
- Model3 is basically model2 but with pooling. This again shows that pooling gives actual accuracy boost. This finding is nothing new. We may use stride sometimes instead of pool to save compute but a lot of times it may result in accuracy loss(Well atleast for small model)

3. Pooling Strategy Comparison (Model0, all ratios)

Comparing all six pooling configurations across Model0 models, averaged over ratio=1/2/3.
Hybrid(Avg→Max) means pool1=AvgPool (attention source), pool2=MaxPool (content). Hybrid(X→∅) means single-pool self-gating.

Pooling Configuration (Model0)	N	Avg Test%	Best%	Worst%	Avg Gap	Avg Params
Hybrid Max→Avg (default)	4	85.29%	86.56%	83.68%	1.85	8.47k
Hybrid Avg→Max (flipped)	4	84.75%	86.87%	81.48%	2.70	9.53k
Hybrid Avg→∅ (self-gate Avg)	3	84.35%	85.87%	82.91%	1.96	8.55k
Hybrid Max→∅ (self-gate Max)	3	83.39%	85.21%	82.07%	2.77	8.55k
AvgPool (plain)	3	84.13%	85.74%	82.77%	2.02	8.35k
MaxPool (plain)	3	82.18%	84.59%	78.81%	2.85	8.35k

Key observations:

- Hybrid(Avg→Max) — flipped configuration where AvgPool generates the attention map and MaxPool provides the content — yields the highest average test accuracy (84.32%) and best single result (86.87%), contrary to the original hypothesis that Max→Avg would be optimal.
- Hybrid(Max→Avg) — the originally hypothesised configuration — ranks second, suggesting both orderings are competitive but the smoother attention map from AvgPool may be less noisy for gating purposes.
- Plain AvgPool outperforms plain MaxPool across all three ratio levels, with a notably lower generalisation gap, indicating MaxPool alone may overfit on sharp local features without the smoothing provided by a second pooling branch.
- Self-gated modes (X→∅) sit between plain pooling and dual-pool hybrids in performance, confirming that the second pooling branch contributes a meaningful signal even though the self-gated variant adds no parameters.

3b. Pooling Strategy Comparison (Model3, fixed ratio)

Pooling Configuration (Model3)	Test%	Gap	Params
Hybrid Max→Avg	83.37%	1.17	3.50k
Hybrid Avg→Max	80.73%	1.98	3.50k
Hybrid Avg→∅	82.19%	1.58	3.50k
Hybrid Max→∅	80.50%	2.12	3.50k
AvgPool	82.12%	1.08	3.40k
MaxPool	80.78%	2.21	3.40k

- In the simpler Model3 architecture, Hybrid(Max→Avg) delivers the best single result (83.37%), reversing the trend seen in Model0 where Avg→Max won.
- MaxPool and Hybrid(Max→∅) produce the widest generalisation gaps in Model3, consistent with MaxPool's overfitting tendency observed in Model0.
- Overall, HybridPooling variants outperform plain pooling in both architectures, validating the pseudo-attention design principle.

4. Channel Expansion Ratio (Model0 only)

Ratio controls the width of intermediate feature representations inside conv_block0 and conv1. Averaged over all six pooling configurations within Model0.

Expansion Ratio (Model0)	N	Avg Test%	Best%	Worst%	Avg Gap	Avg Params
Ratio = 1 (narrow)	6	81.95%	83.68%	78.81%	1.84	4.87k
Ratio = 2 (default)	7	84.40%	86.56%	82.89%	2.19	8.19k
Ratio = 3 (wide)	7	85.68%	86.87%	84.59%	2.94	12.40k

Key observations:

- Higher ratio consistently improves both training and test accuracy — ratio=3 models average 85.72% test accuracy versus 82.40% at ratio=1. This was the expected outcome so nothing fancy
- The generalisation gap also widens slightly with ratio, from 1.56 pp at ratio=1 to 2.41 pp at ratio=3, indicating a mild increase in overfitting with larger capacity. This was also expected because In larger model the the gap between training accuracy and test accuracy usually widen
- Ratio=2 offers the best compromise between accuracy and gap: close to ratio=3 performance (84.42%) with a tighter gap.
- The parameter cost of ratio=3 is roughly 2.5× that of ratio=1 within Model0, so the accuracy gain of approximately 3.3 pp comes at a substantial computational cost.

5. Efficiency — Accuracy per Parameter (Top 10)

Ranked by test accuracy divided by parameter count (k). Higher values indicate better accuracy per unit of model size. Green row = most efficient model overall.

Model	Arch	Pooling	Ratio	Params(k)	MMA C	TestAcc%	Acc /Param
model4_0	Model 2	Strided Conv	1	3.15	9.25	77.38%	24.57
model23_0	Model 3	AvgPool	-	3.4	24.14	82.12%	24.15
model22_0	Model 3	Hybrid(Max→Avg)	-	3.5	25.07	83.37%	23.82
model24_0	Model 3	MaxPool	-	3.4	24.14	80.78%	23.76
model26_0	Model 3	Hybrid(Avg→∅)	-	3.5	24.32	82.19%	23.48
model25_0	Model 3	Hybrid(Avg→Max)	-	3.5	25.07	80.73%	23.07
model27_0	Model 3	Hybrid(Max→∅)	-	3.5	24.32	80.5%	23
model8_0	Model 0	AvgPool	1	4.79	25.62	82.77%	17.28

model5_0	Model 0	Hybrid(Max→Avg)	1	4.91	26.66	83.68%	17.04
model2_0	Model 1	Strided Conv	1	4.79	9.95	81.37%	16.99

Key observations:

- Model3 variants dominate the top of the efficiency ranking despite having no channel expansion, because their parameter counts (3.4–3.5k) are among the lowest in the experiment.
- model22_0 (Model3 + Hybrid Max→Avg) achieves the best accuracy-per-parameter ratio, combining Model3's compactness with the strongest pooling configuration for that architecture.
- Among Model0 variants, model5_0 (ratio=1, Hybrid Max→Avg) offers the best efficiency, demonstrating that even with the richer Model0 structure, ratio=1 is competitive when parameters are scarce.

6. Overall Rankings

Summary of top and bottom performers across key metrics.

Metric	Model
Highest test accuracy	model15_0 — 86.87% (Model0, Hybrid Avg→Max, ratio=3, BS=32)
Lowest test accuracy	model4_0 — 77.38% (Model2, strided conv, ratio=1)
Smallest gap	model0_1 — 2.11 pp (BS=16 variant of model0)
Largest gap	model3_0 — 3.61 pp (Model1, ratio=3)
Best accuracy/param	model22_0 — 23.82 (Model3, Hybrid Max→Avg)
Best accuracy/MMAC	model22_0 — 3.32 (Model3, Hybrid Max→Avg)
Lightest model	model4_0 — 3.15k params
Heaviest model	model15_0 / model6_0 / model15_1 / model18_0 / model21_0 — 12.48k params

Note on batch size variants — model0_1 and model15_1 are identical to model0_0 and model15_0 respectively, retrained with batch size 16 instead of 40. For model0, BS=16 improved test accuracy by +1.08 pp (85.48% to 86.56%) and narrowed the generalisation gap. For model15 — the overall best model — BS=16 marginally degraded test accuracy by 0.81 pp (86.87% to 86.06%) and widened the gap from 2.78 pp to 3.63 pp, suggesting larger-capacity models are more sensitive to batch noise. Both BS=16 variants are excluded from grouped averages to avoid double-counting.

2.2 Cifar 10

Goal

This project shifts focus from using a filter stack, as in the previous Cat Vs Dog project, to an iterative process of experimentation. The approach involves testing diverse model configurations through trial and error, systematically retaining effective components and replacing or removing those that prove ineffective. Important thing to note here is that we are taking the best test accuracy for the comparison, which has its caveats, because at the end of training the accuracy oscillates i.e it can happen that some model got lucky. I dont have enough compute to test it with different seeds so we have to deal with the caveats for now

Training Details

Parameter	Value / Detail
Epochs	60
Optimiser	AdamW
Learning rate — Epochs 1–31	0.01
Learning rate — Epochs 32–51	0.001
Learning rate — Epoch 52-60	0.0001
Batch size	64 and 128
Input image size	32 × 32 pixels
Augmentation	Random horizontal flip and random crop applied to the train data
Weight initialisation	Kaiming Normal (He) applied to all Conv2d and Linear layers — mode=fan_in, negative slope a=0.25 (matching PReLU default). Biases initialised to zero.
Initialisation rationale	Kaiming Normal with a=0.25 accounts for the PReLU activation used throughout the network, which behaves as a leaky ReLU with a learnable slope initialised near 0.25. This keeps gradient variance stable at network initialisation.

Architecture Description

BaseModelF32

Forward Pass

INPUT <i>Tensor of shape [B, 3, 32, 32] — CIFAR-10 RGB images</i>
– B = batch size. Channels = 3 (RGB). Spatial dimensions fixed at 32 × 32 pixels, as defined by the CIFAR-10 dataset.
– Channels are normalised before entering the network (Batchnorm is used. Which is unusual, but the empirical result for my case suggested it works. Although It kinda makes sense if you think about it)



Block 1 *(task-specific, injected at construction)*

An arbitrary nn.Module operating on the tensor produced by the previous block. Each block may perform any combination of convolution, normalisation, activation, pooling, attention, or recurrent operations. The only contract is that it accepts and returns a tensor compatible with the next block's expected input shape.



Block 2 *(task-specific, injected at construction)*

An arbitrary nn.Module operating on the tensor produced by the previous block. Each block may perform any combination of convolution, normalisation, activation, pooling, attention, or recurrent operations. The only contract is that it accepts and returns a tensor compatible with the next block's expected input shape.



Block 3 *(task-specific, injected at construction)*

An arbitrary nn.Module operating on the tensor produced by the previous block. Each block may perform any combination of convolution, normalisation, activation, pooling, attention, or recurrent operations. The only contract is that it accepts and returns a tensor compatible with the next block's expected input shape.



Block 4 (task-specific, injected at construction)

An arbitrary nn.Module operating on the tensor produced by the previous block. Each block may perform any combination of convolution, normalisation, activation, pooling, attention, or recurrent operations. The only contract is that it accepts and returns a tensor compatible with the next block's expected input shape.



FINAL HEAD *Flatten* → *Linear(240, 10)*

- Flatten: collapses all spatial and channel dimensions into a single vector. The four blocks are expected to collectively produce a tensor whose total non-batch element count equals 240.
- Linear(240, 10): fully-connected projection from the 240-dimensional feature vector to 10 output logits, one per class.
- No activation is applied after the linear layer — raw logits are returned, compatible with nn.CrossEntropyLoss which applies softmax internally.



OUTPUT *Logit vector of shape [B, 10]*

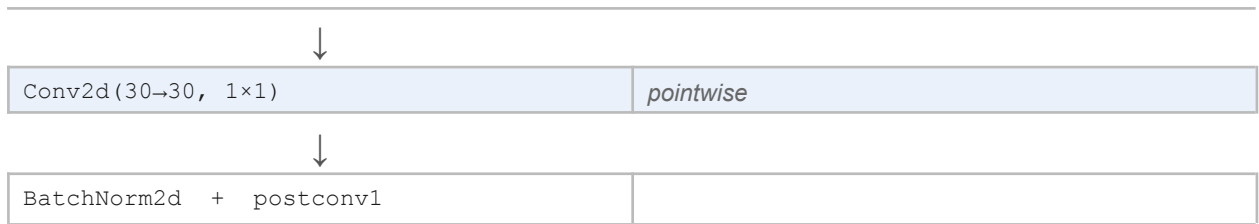
- 10 raw class scores per sample, corresponding to: airplane (0), automobile (1), bird (2), cat (3), deer (4), dog (5), frog (6), horse (7), ship (8), truck (9).
- The predicted class is argmax over dimension 1. Loss is computed externally (typically CrossEntropyLoss), not inside the model.

Block 1 — Variants

Input: [B, 3, 32, 32]. All variants expand 3 → 30 channels using depthwise convolutions. postconv1 and postconv2 are injected parameters; postconv2 performs spatial downsampling.

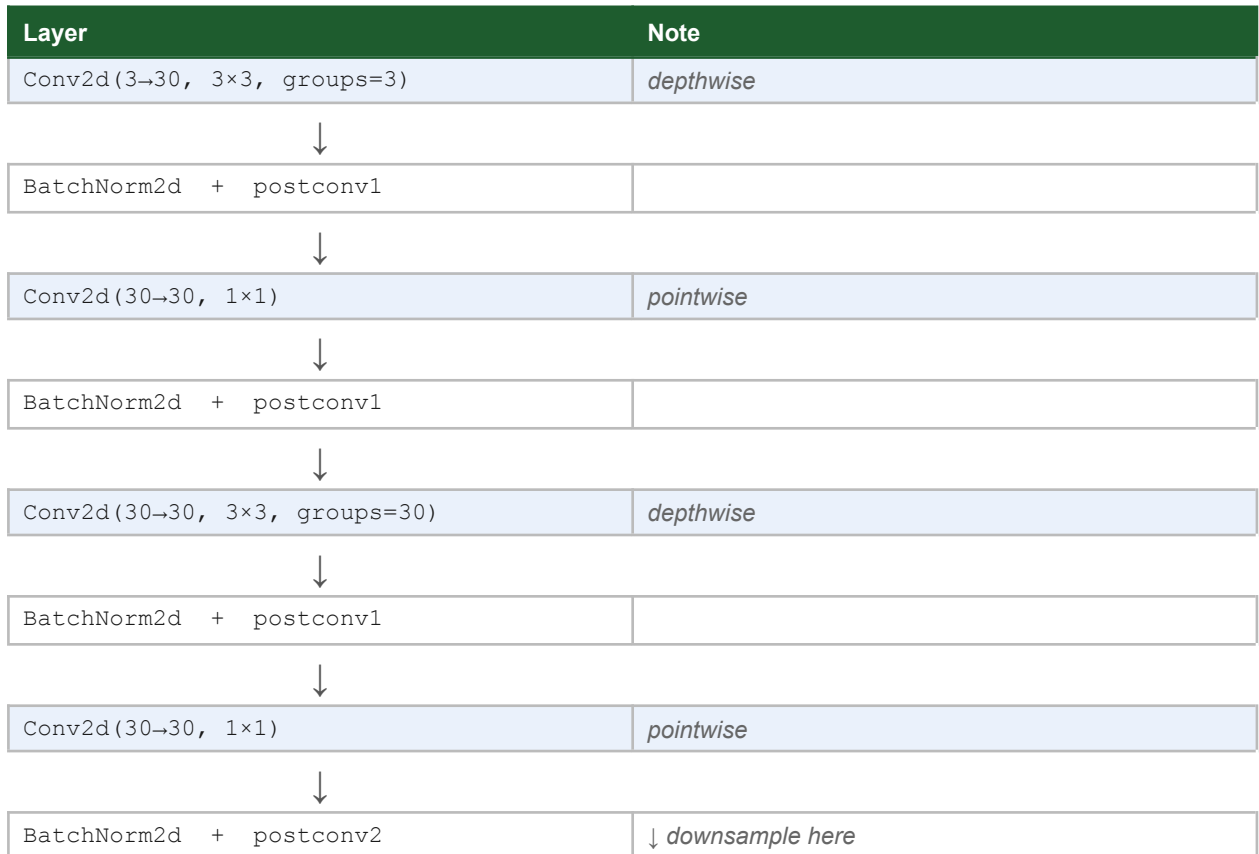
Block1 V0

Layer	Note
Conv2d(3→30, 3×3, groups=3)	<i>depthwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(30→30, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(30→30, 3×3, groups=30)	<i>depthwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>



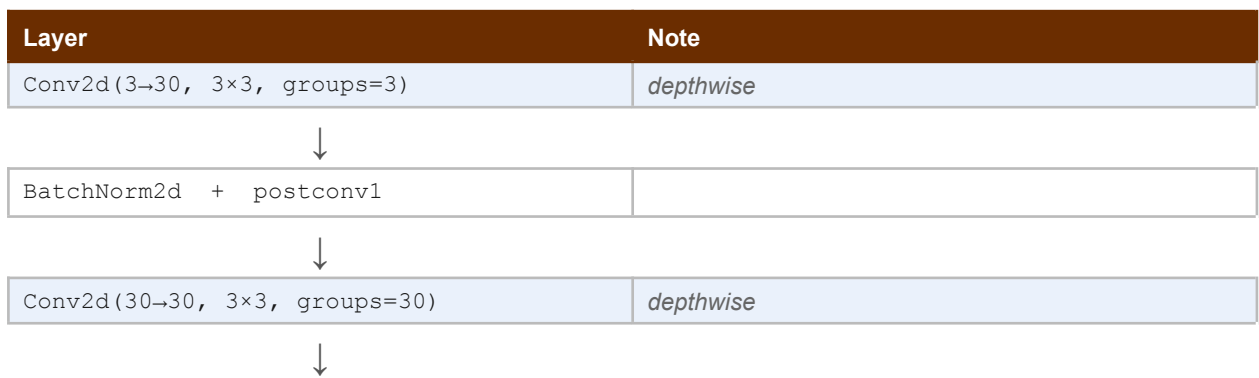
Block1 V1

Same as V0 — only difference is postconv2 moved to the end.



Block1 V2

Pointwise convolutions removed. Depthwise only — no cross-channel mixing.



BatchNorm2d + postconv2	↓ <i>downsample here</i>
-------------------------	--------------------------

Block 2 — Variants

Block2 V0

Layer	Note
Conv2d(30→60, 3×3, groups=30)	<i>depthwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 3×3, groups=60)	<i>depthwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>
↓	
Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	

Block2 V1

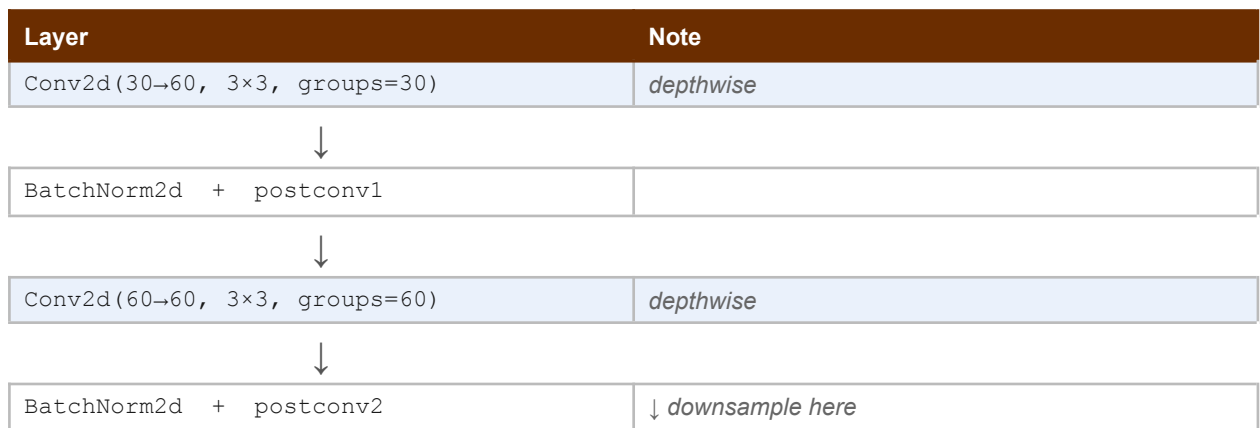
Same as V0 — only difference is postconv2 moved to the end.

Layer	Note
Conv2d(30→60, 3×3, groups=30)	<i>depthwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 3×3, groups=60)	<i>depthwise</i>



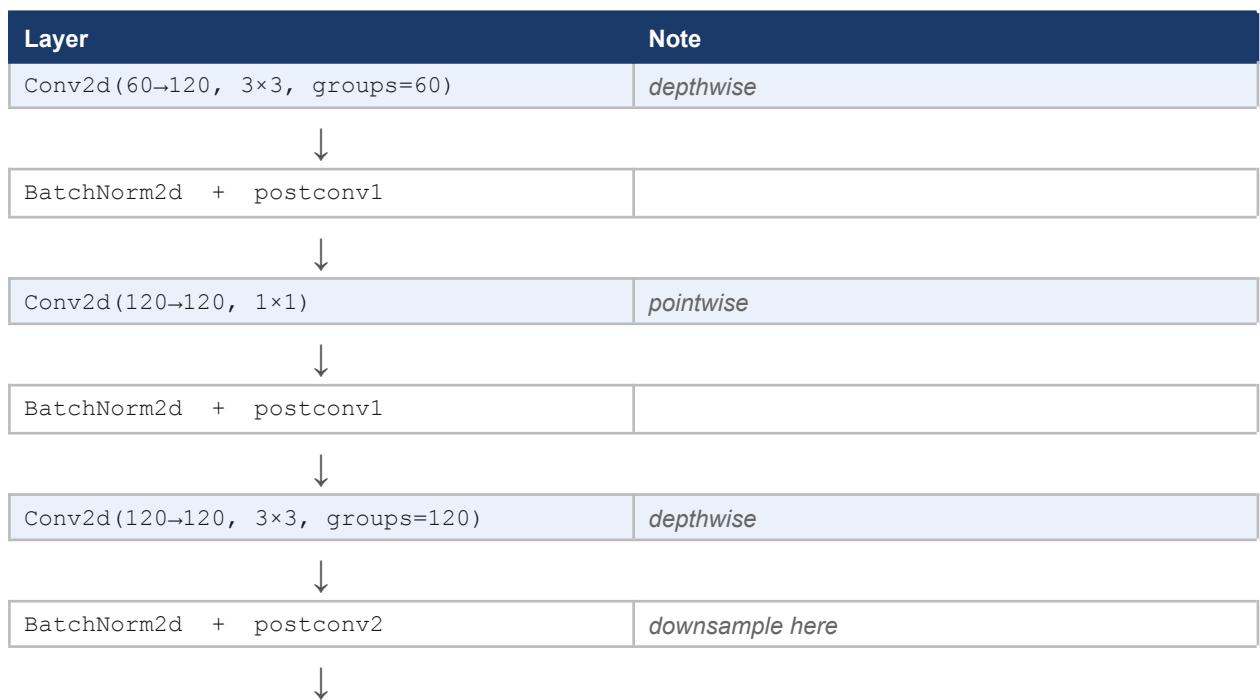
Block2 V2

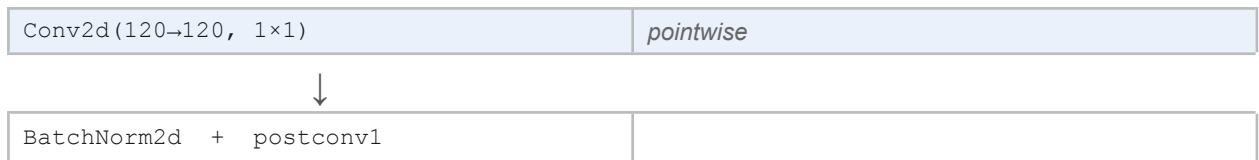
Pointwise convolutions removed. Depthwise only — no cross-channel mixing.



Block 3 — Variants

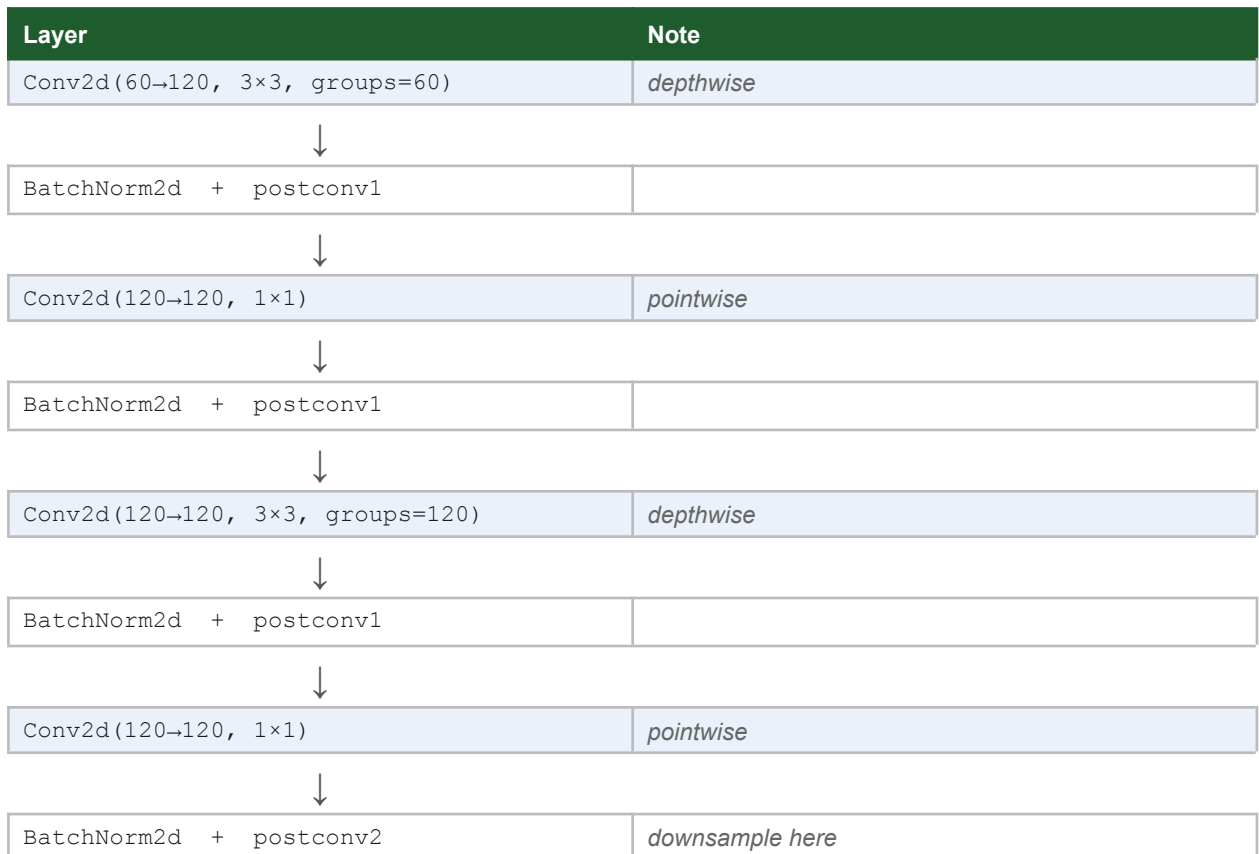
Block3 V0





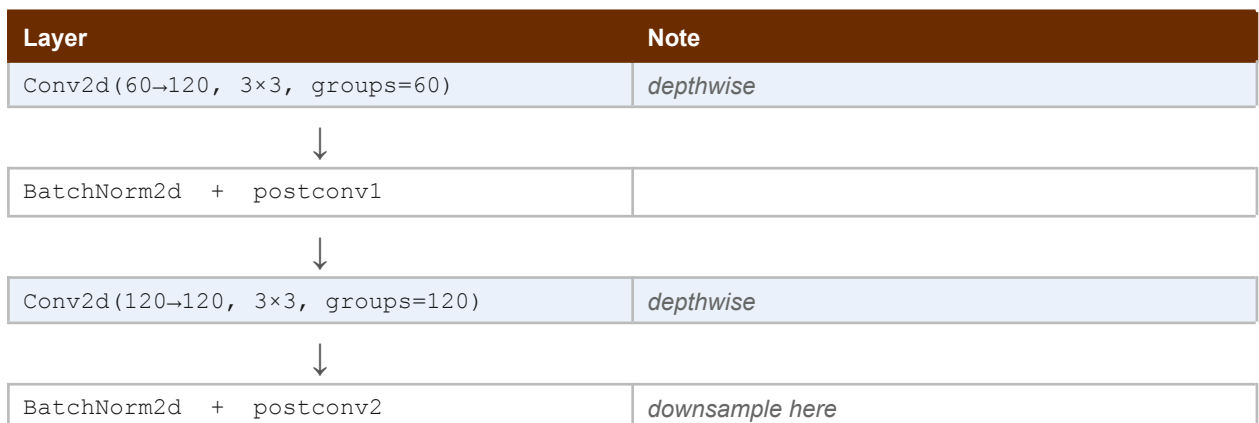
Block3 V1

Same as V0 — the only difference is that postconv2 is moved to the end.

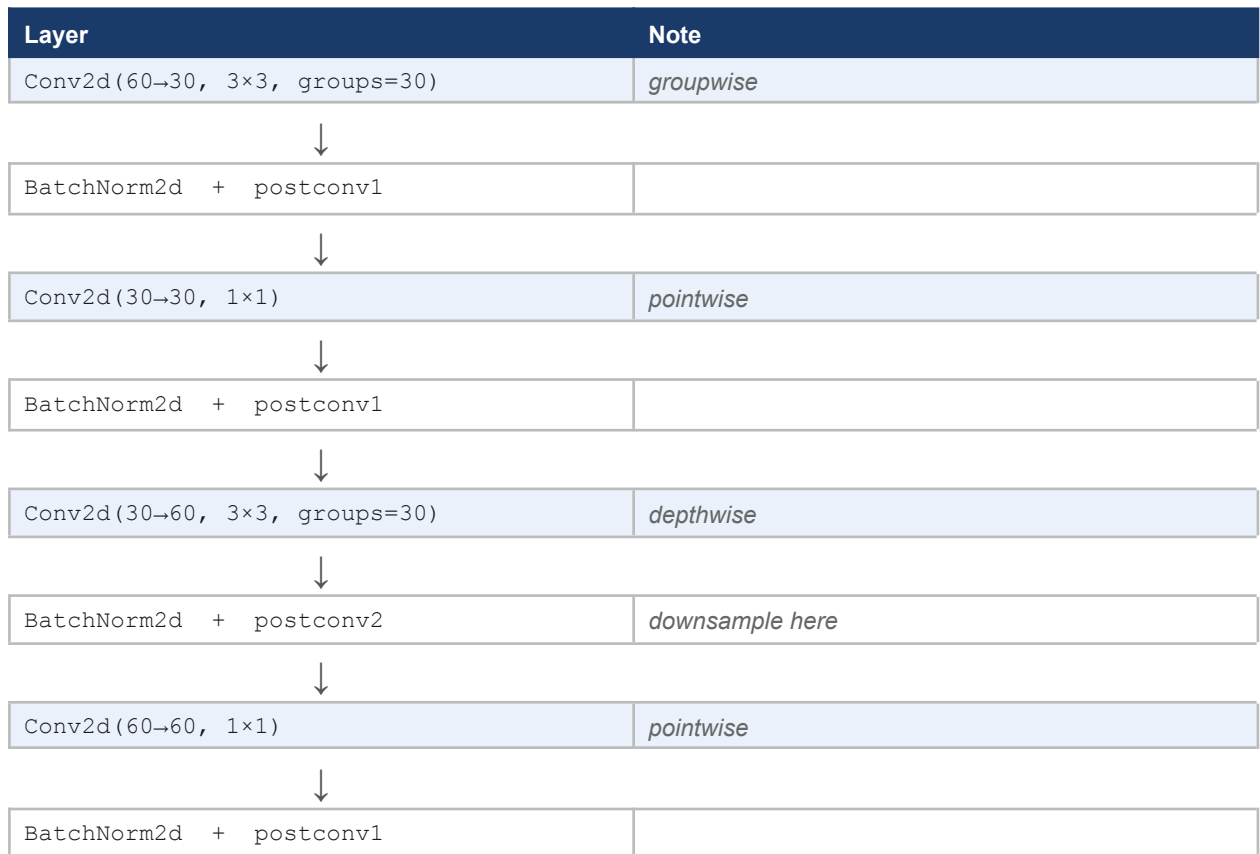


Block3 V2

Pointwise convolutions removed. Depthwise only — no cross-channel mixing.

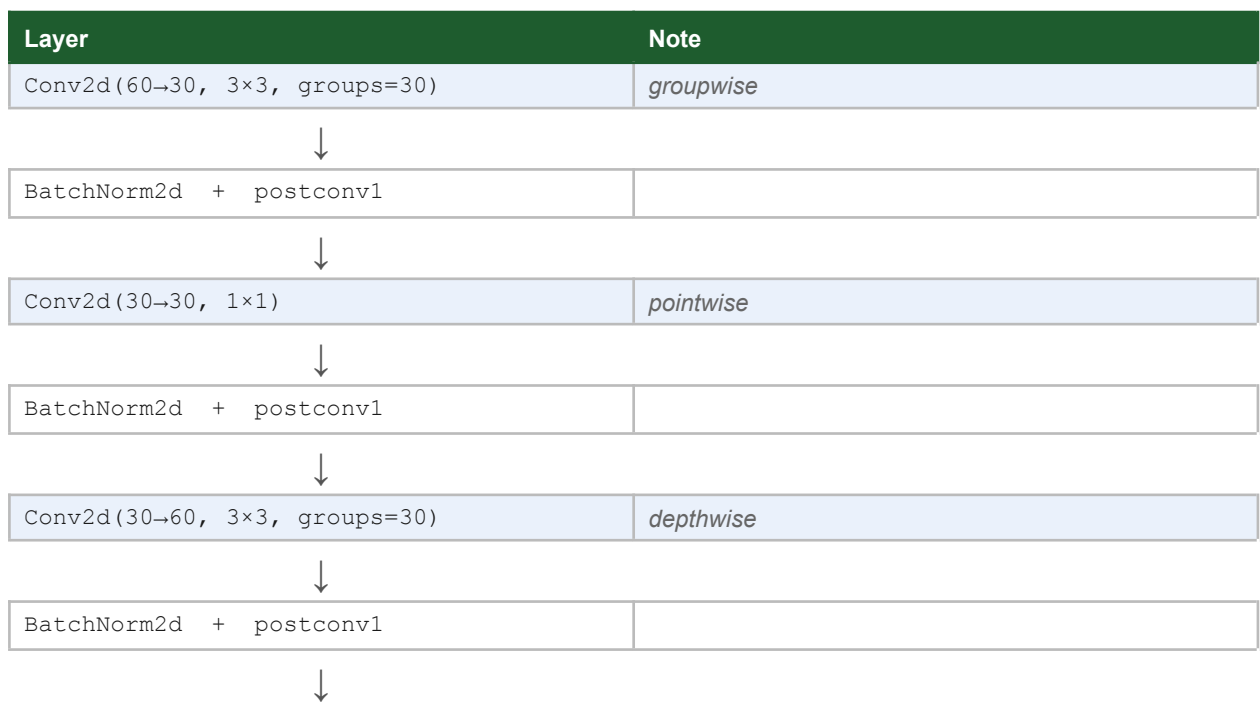


Block3 V3



Block3 V4

Same as V3 — the only difference is that postconv2 is moved to the end.



Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>

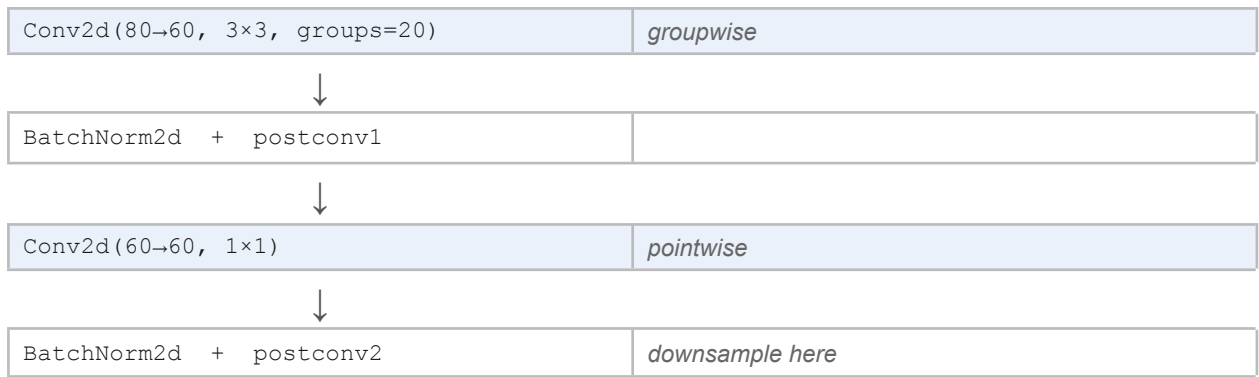
Block3 V5

Layer	Note
Conv2d(60→80, 3×3, groups=20)	<i>groupwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(80→80, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(80→60, 3×3, groups=20)	<i>groupwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>
↓	
Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	

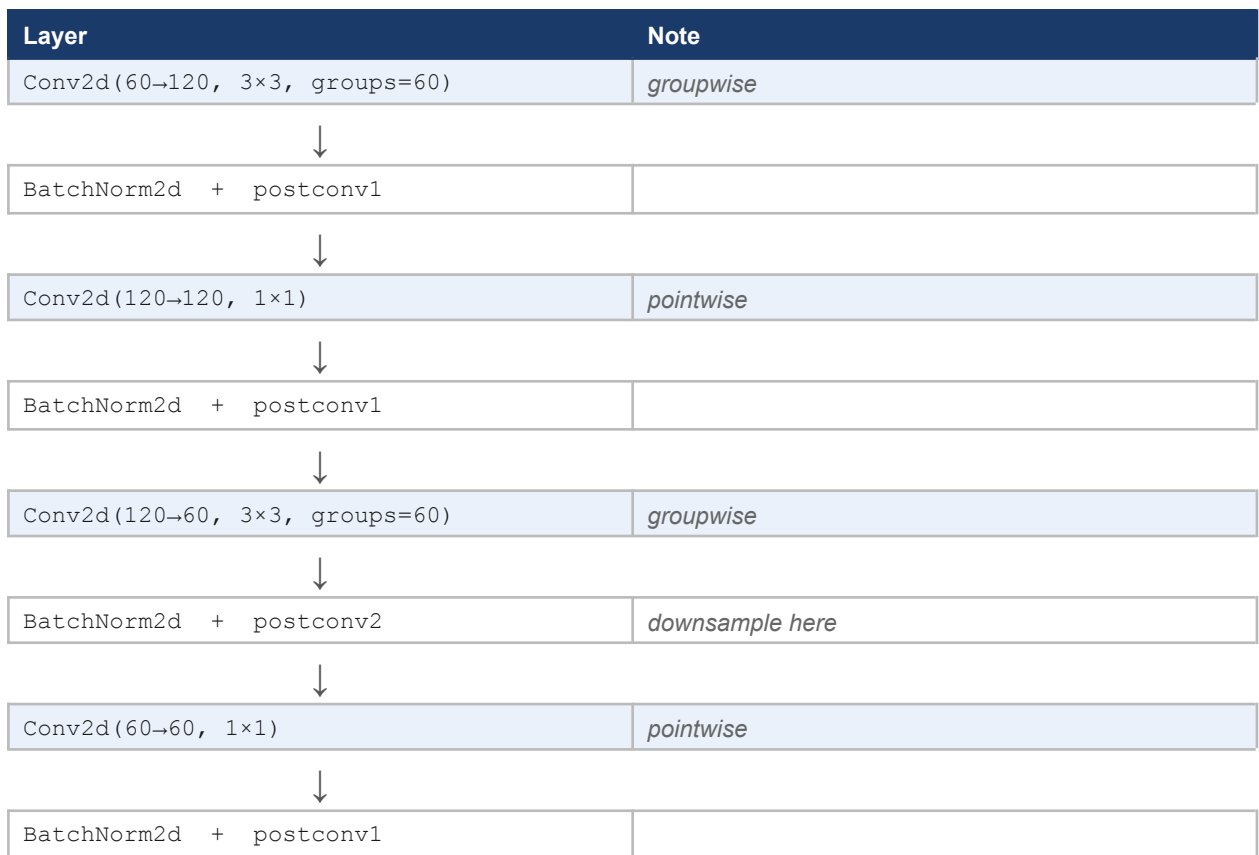
Block3 V6

Same as V5 — the only difference is that postconv2 is moved to the end.

Layer	Note
Conv2d(60→80, 3×3, groups=20)	<i>groupwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(80→80, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	
↓	

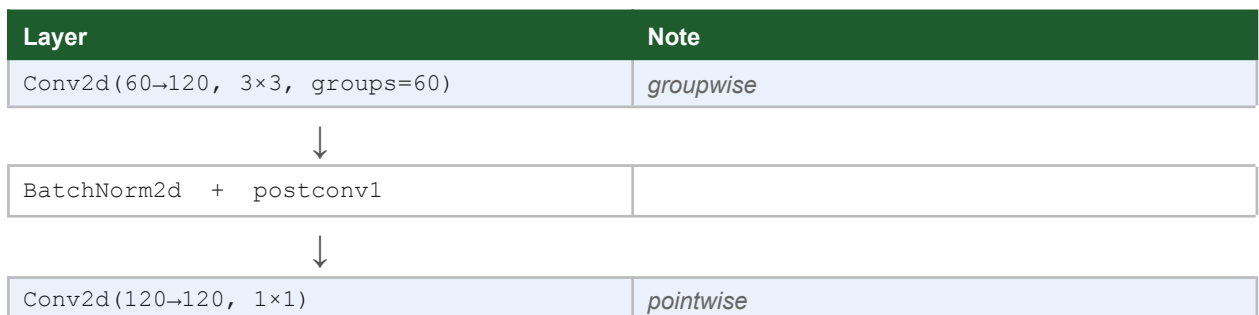


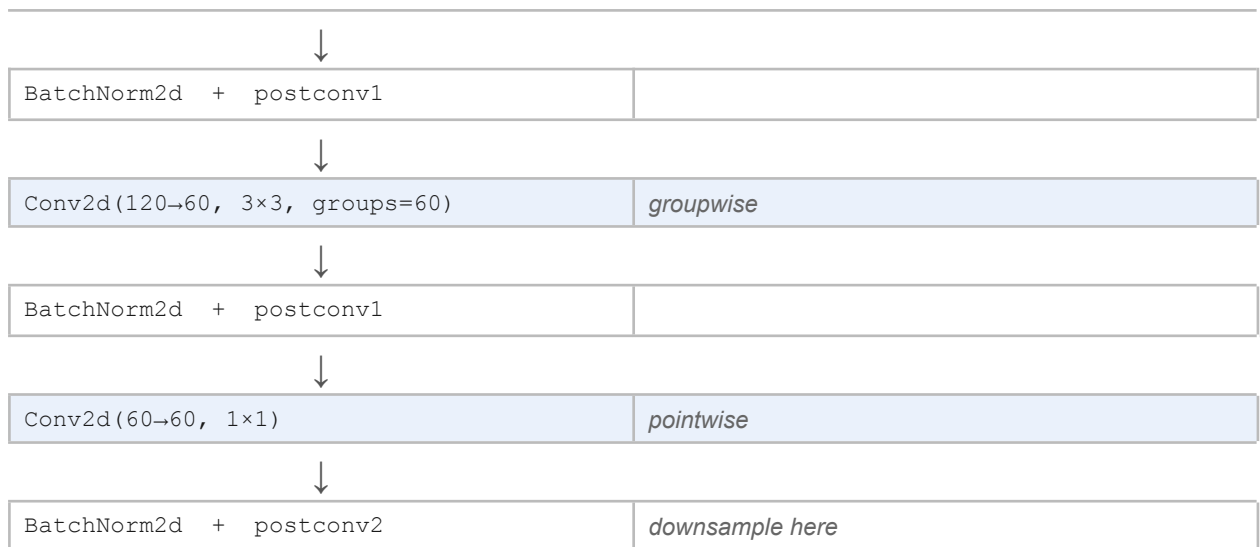
Block3 V7



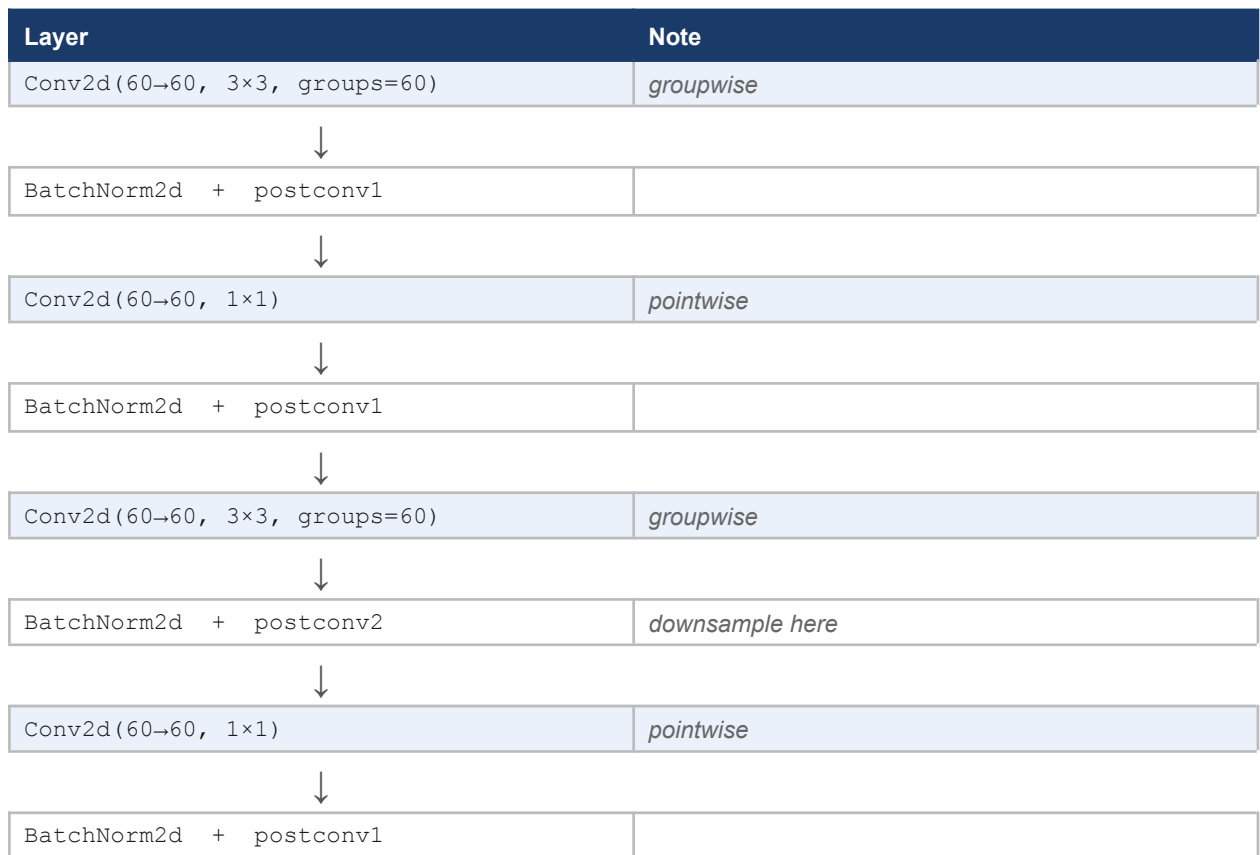
Block3 V8

Same as V7 — the only difference is that postconv2 is moved to the end.





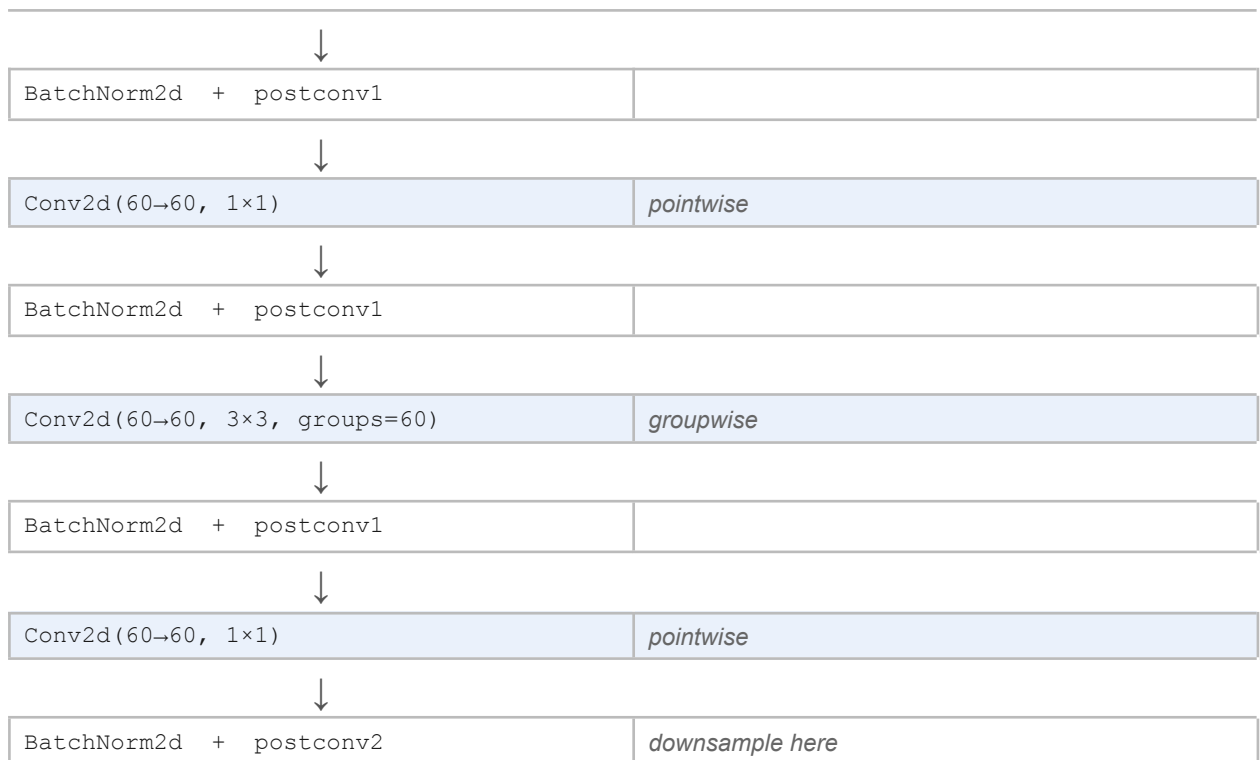
Block3 V9



Block3 V10

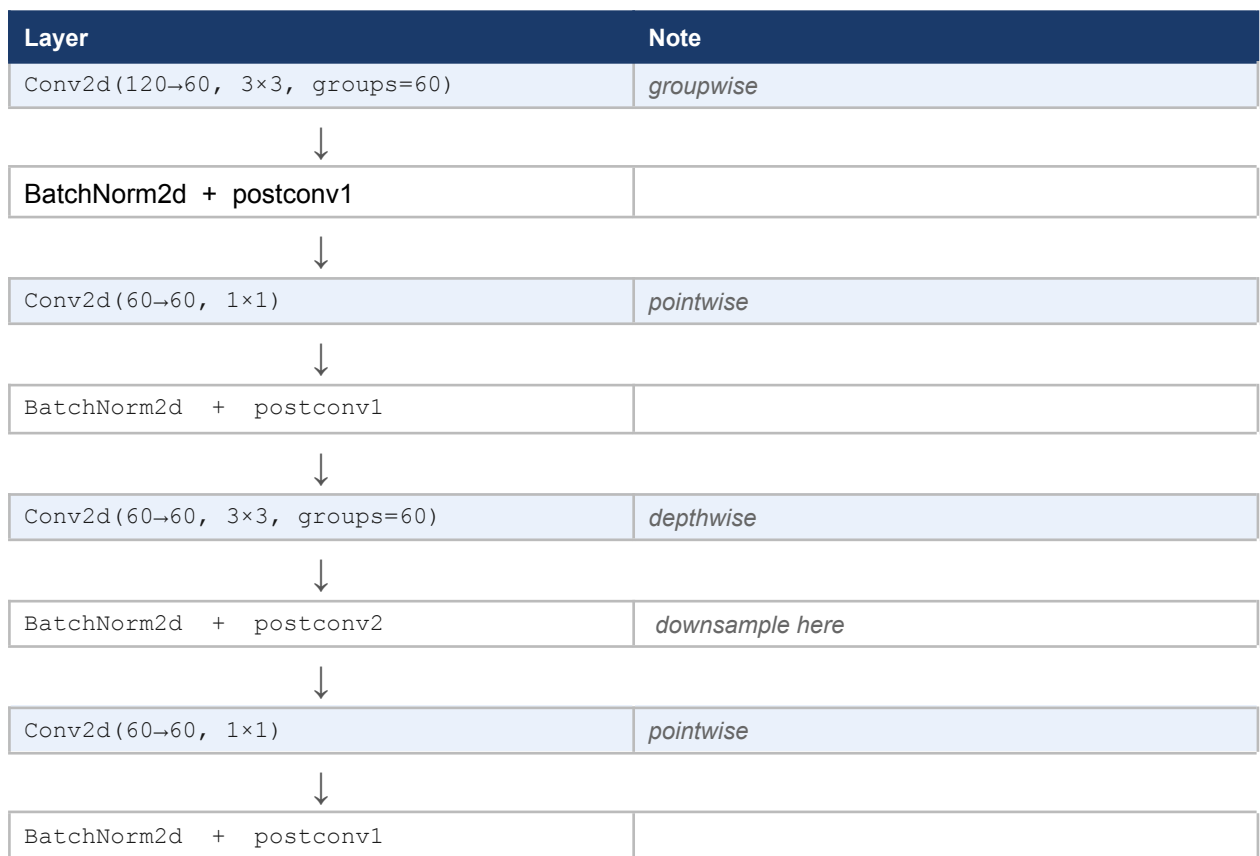
Same as V9 — the only difference is that postconv2 is moved to the end.

Layer	Note
Conv2d(60→60, 3×3, groups=60)	groupwise



Block 4 — Variants

Block4 V0



Block4 V1

Same as V0 — only difference is postconv2 moved to the end.

Layer	Note
Conv2d(120→60, 3×3, groups=60)	<i>groupwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 3×3, groups=60)	<i>depthwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 1×1)	<i>pointwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>

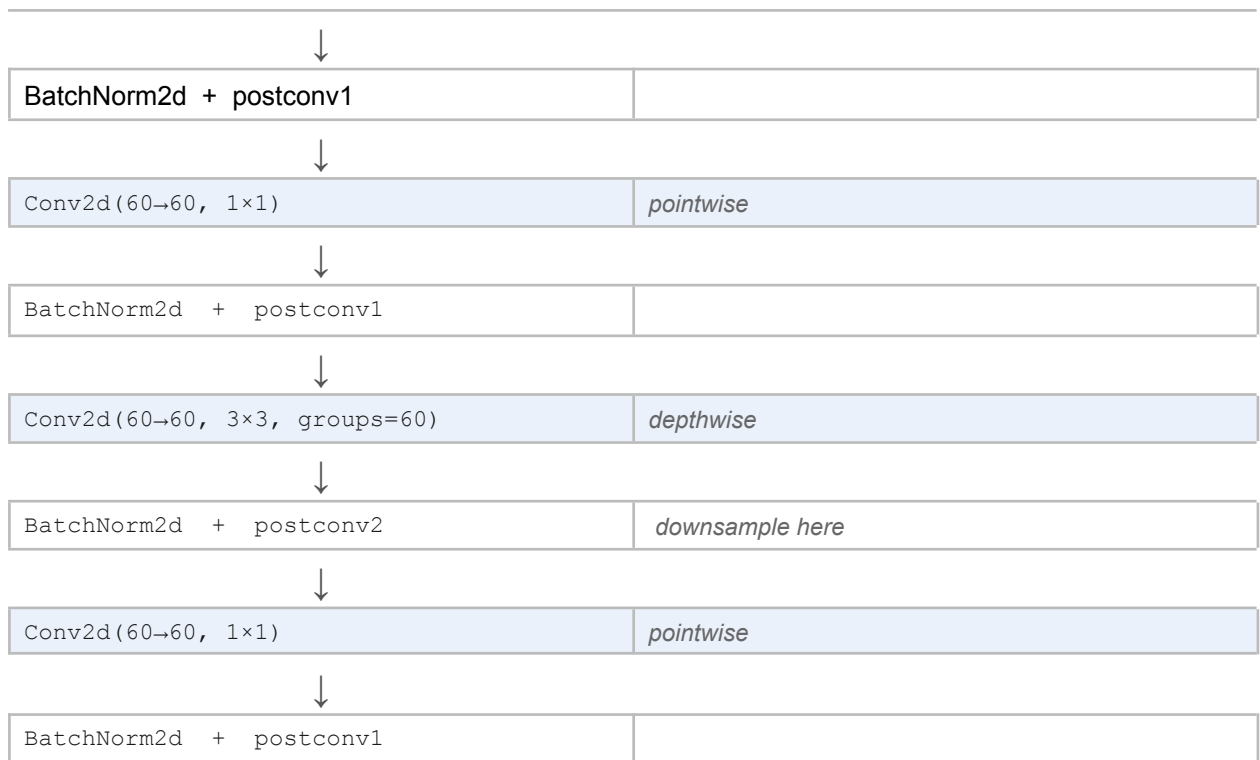
Block4 V2

Pointwise convolutions removed. Depthwise only — no cross-channel mixing.

Layer	Note
Conv2d(120→60, 3×3, groups=60)	<i>groupwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 3×3, groups=60)	<i>depthwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>

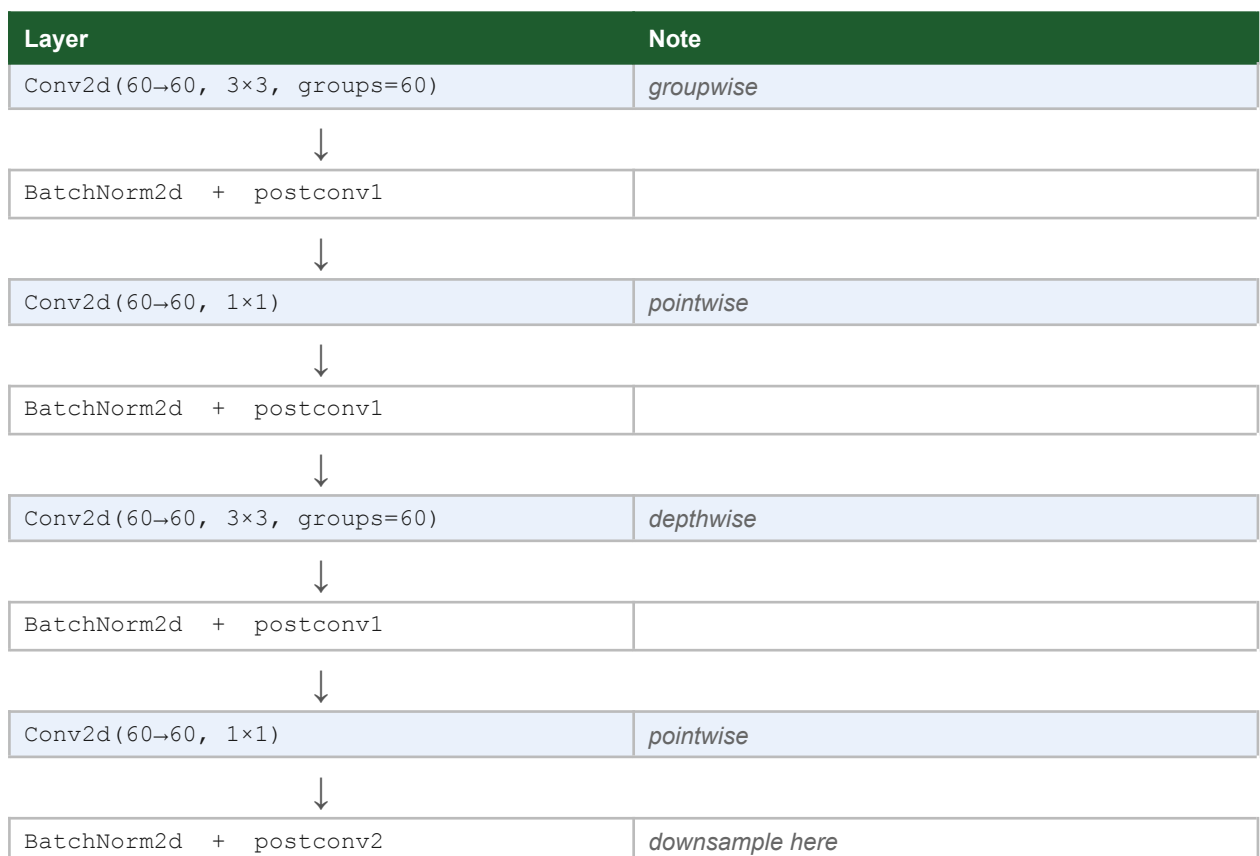
Block4 V3

Layer	Note
Conv2d(60→60, 3×3, groups=60)	<i>groupwise</i>



Block4 V4

Same as V3 — only difference is postconv2 moved to the end.



Block4 V5

Pointwise convolutions removed. Depthwise only — no cross-channel mixing.

Layer	Note
Conv2d(120→60, 3×3, groups=60)	<i>groupwise</i>
↓	
BatchNorm2d + postconv1	
↓	
Conv2d(60→60, 3×3, groups=60)	<i>depthwise</i>
↓	
BatchNorm2d + postconv2	<i>downsample here</i>

postconv Variants

postconv1 — activation only, no spatial change

Name	Composition
postconv1_A	PReLU()
postconv1_B	ReLU()

postconv2 — activation + spatial downsampling

Name	Composition
postconv2_A	PReLU() → HybridPoolingV0(Max, Avg)
postconv2_B	PReLU() → HybridPoolingV0(Avg, Max)
postconv2_C	PReLU() → MaxPool2d(2)
postconv2_D	ReLU() → HybridPoolingV0(Max, Avg)
postconv2_E	ReLU() → HybridPoolingV0(Avg, Max)
postconv2_F	ReLU() → MaxPool2d(2)
postconv2_G	HybridPoolingV0(Max, Avg) → PReLU()
postconv2_H	HybridPoolingV0(Avg, Max) → PReLU()
postconv2_I	MaxPool2d(2) → PReLU()
postconv2_J	HybridPoolingV0(Max, Avg) → ReLU()
postconv2_K	HybridPoolingV0(Avg, Max) → ReLU()
postconv2_L	MaxPool2d(2) → ReLU()

Model Summery(TODO:Have to fill it up)

Name	Block1	Block2	Block3	Block4	Postconv1	Postconv2
Model0						
Model1						
Model2						
Model3						
Model4						
Model5						
Model6						
Model7						
Model8						
Model9						
Model10						
Model11						
Model12						
Model13						
Model14						
Model15						
Model16						
Model19						
Model20						
Model21						
Model22						
Model23						
Model24						
Model25						
Model26						
Model27						

Model28						
Model29						
Model30						
Model31						
Model32						
Model33						
Model34						
Model35						
Model36						
Model37						
Model38						
Model39						
Model40						
Model41						
Model42						
Model43						

TODO:FILL IT UP

Name	Train Accuracy		Test Accuracy		Param count	MMAC
	64 batch	128 batch	64 batch	128 batch		
Model0						
Model1						
Model2						
Model3						
Model4						
Model5						
Model6						
Model7						
Model8						

Model9						
Model10						
Model11						
Model12						
Model13						
Model14						
Model15						
Model16						
Model19						
Model20						
Model21						
Model22						
Model23						
Model24						
Model25						
Model26						
Model27						
Model28						
Model29						
Model30						
Model31						
Model32						
Model33						
Model34						
Model35						
Model36						
Model37						
Model38						
Model39						

Model40						
Model41						
Model42						
Model43						